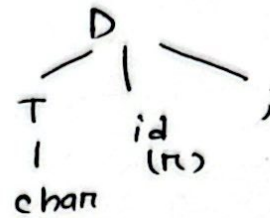
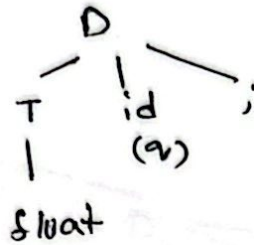
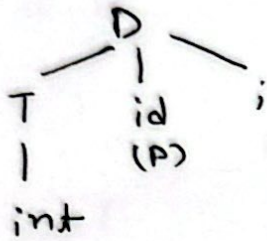


Declaration and Type expression grammar $D \rightarrow T \text{ id};$ $T \rightarrow \text{int} \mid \text{float} \mid \text{char}$

```

int p;
float q;
char n;

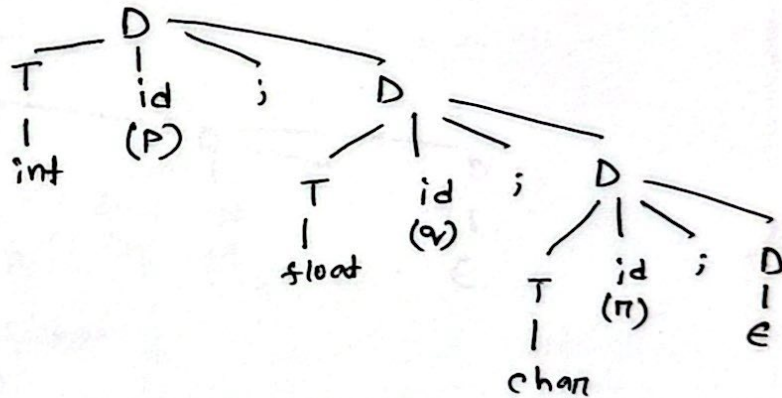
```



Source code এর corresponding অবস্থান- 1 টি parse tree হবে।

এখানে 3 টি declaration এর জন্য 3 টি parse tree generate হয়েছে। কিন্তু 3 টি declaration 1 টি parse tree র under এ আকা উচিত। কারণ compiler 1 টি ই parse tree generate করে।

Main goal: 1 টি parse tree র under এ সব declaration রাখা

 $D \rightarrow T \text{ id}; D \mid \epsilon$ $T \rightarrow \text{int} \mid \text{float} \mid \text{char}$ 

```

int p;
float q;
char n;

```

Azmari Sultana

Nitrin[®] SR
Nitroglycerin USP 2.6 mg & 6.4 mg Tablet


Healthcare

VESTAR MR[®]
TRIMETAZIDINE

*

$D \rightarrow T N ; D$

$T \rightarrow \text{int} | \text{float} | \text{char}$

$D \rightarrow \epsilon$

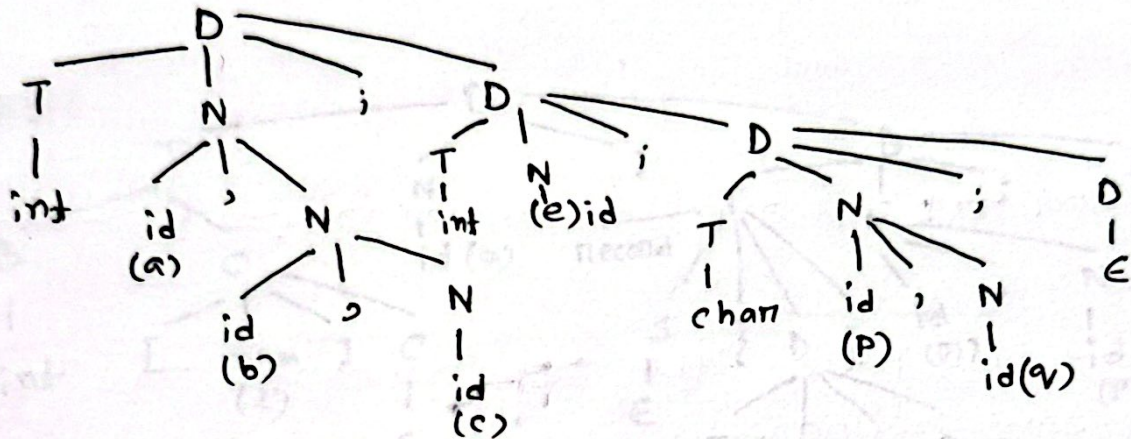
$N \rightarrow \text{id}$

$N \rightarrow \text{id}, N$

`int a, b, c;`

`int e;`

`char p, q;`



*

$D \rightarrow T N ; D | \epsilon$

$T \rightarrow B C$

$B \rightarrow \text{int} | \text{float} | \text{char}$

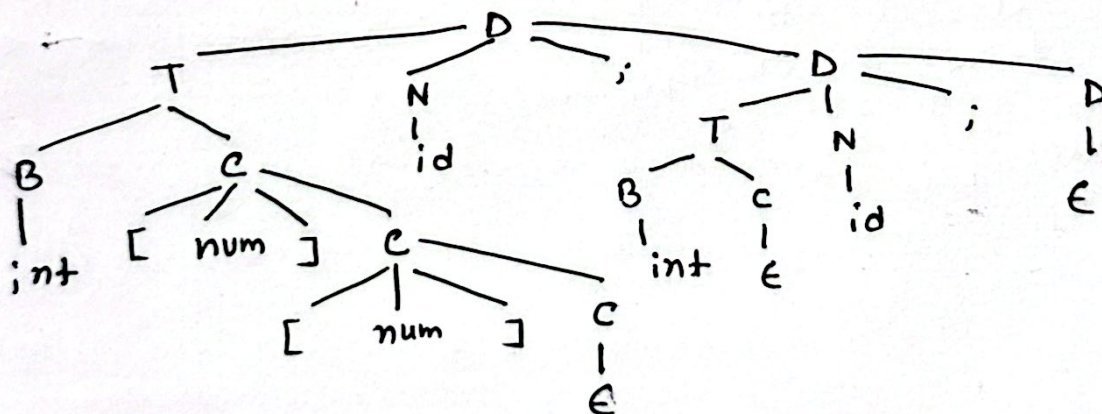
$C \rightarrow [\text{num}] C_1 | \epsilon$

$N \rightarrow \text{id} | \text{id}, N$

input string:

`int [2][3] a;`

`int b;`



*

input :

$D \rightarrow T N ; D | \epsilon$

$T \rightarrow B C | \text{recond } S \{ ' D ' \}$

$S \rightarrow \text{id} | \epsilon$

$B \rightarrow \text{int} | \text{float} | \text{char}$

$C \rightarrow [\text{num}] C, | \epsilon$

$N \rightarrow \text{id} | \text{id}, N$

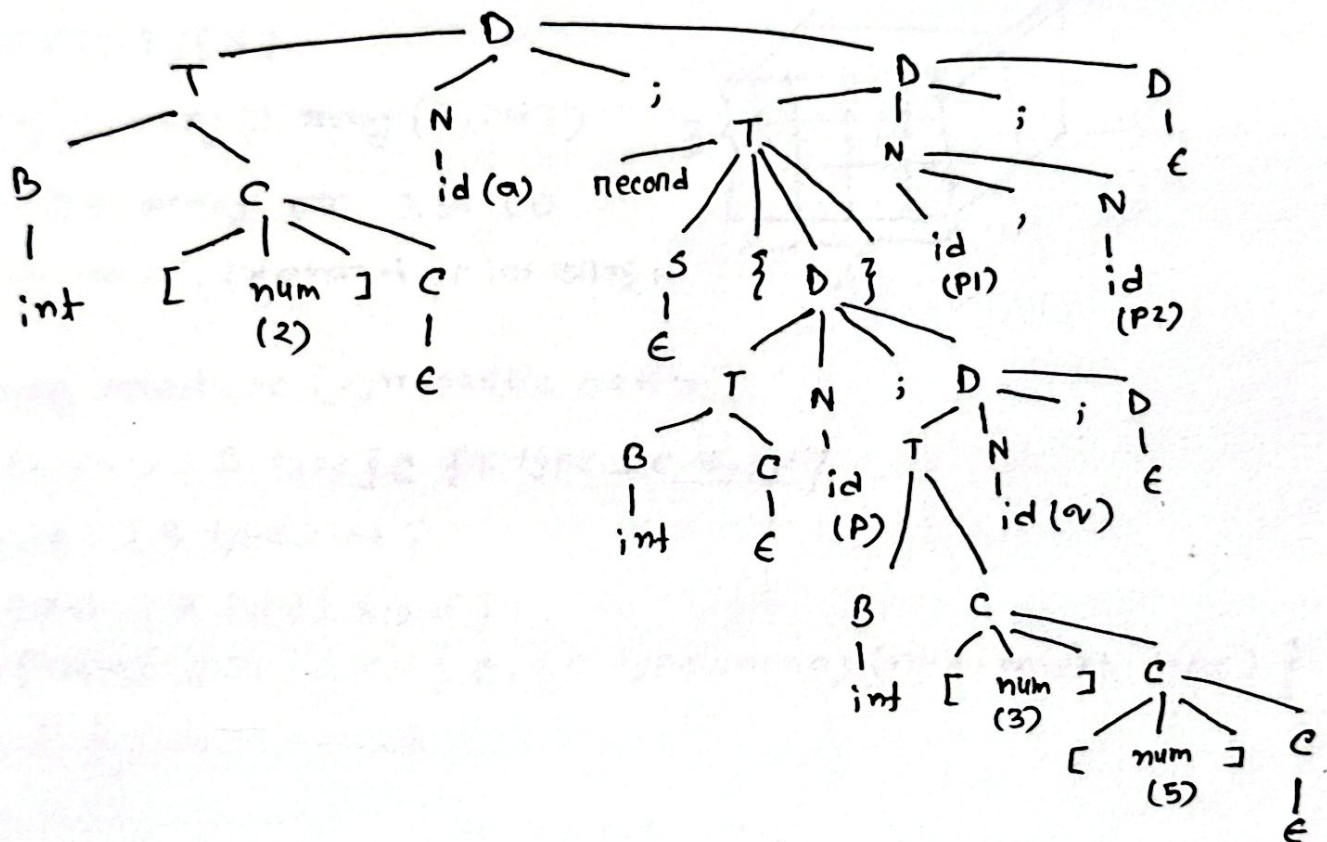
`int[2] a;`

`recond {`

`int p;`

`int[3][5] q;`

`} p1, p2;`

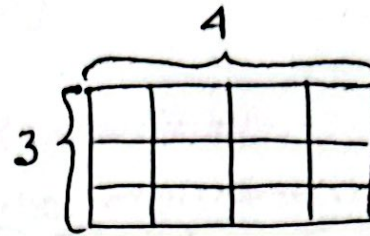


SDT for Array Type Calculation

Lecture 12

`int [3] [4]`

$\hookrightarrow \text{array}(3, \text{array}(4, \text{int}))$



↓
2D array ত 3 টা

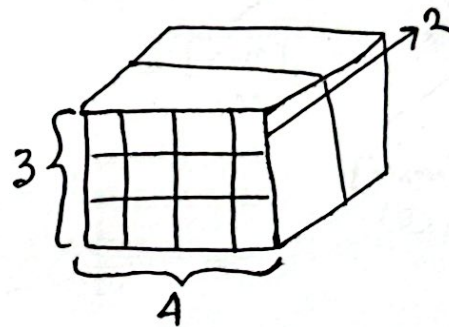
1D array আছে।

প্রত্যেকটি 1D array ত 4 টা
করে integer থাকতে পারে।

`int [2] [3] [4]`

$\hookrightarrow \text{array}(2, \text{array}(3, \text{array}(4, \text{int})))$

২ টা 2D array ত 3 টা 1D
array আছে, (যেখানে 4 টা int আছে।



* Using attribute (syntactic action)

$T \rightarrow B \{ C.i = B.type \} C \{ T.type = C.type \}$

$B \rightarrow \text{int} \{ B.type = \text{int} \}$

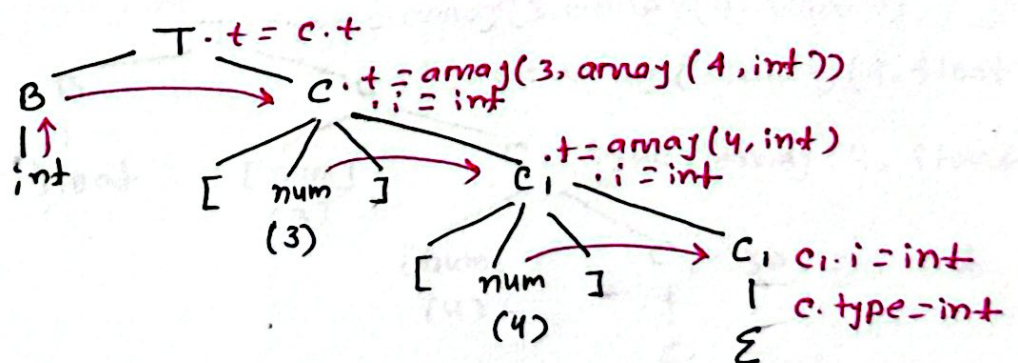
$B \rightarrow \text{float} \{ B.type = \text{float} \}$

$C \rightarrow [num] \{ C.i = C.i \} C_1 \{ C.type = \text{array}(num.val, C_1.type) \}$

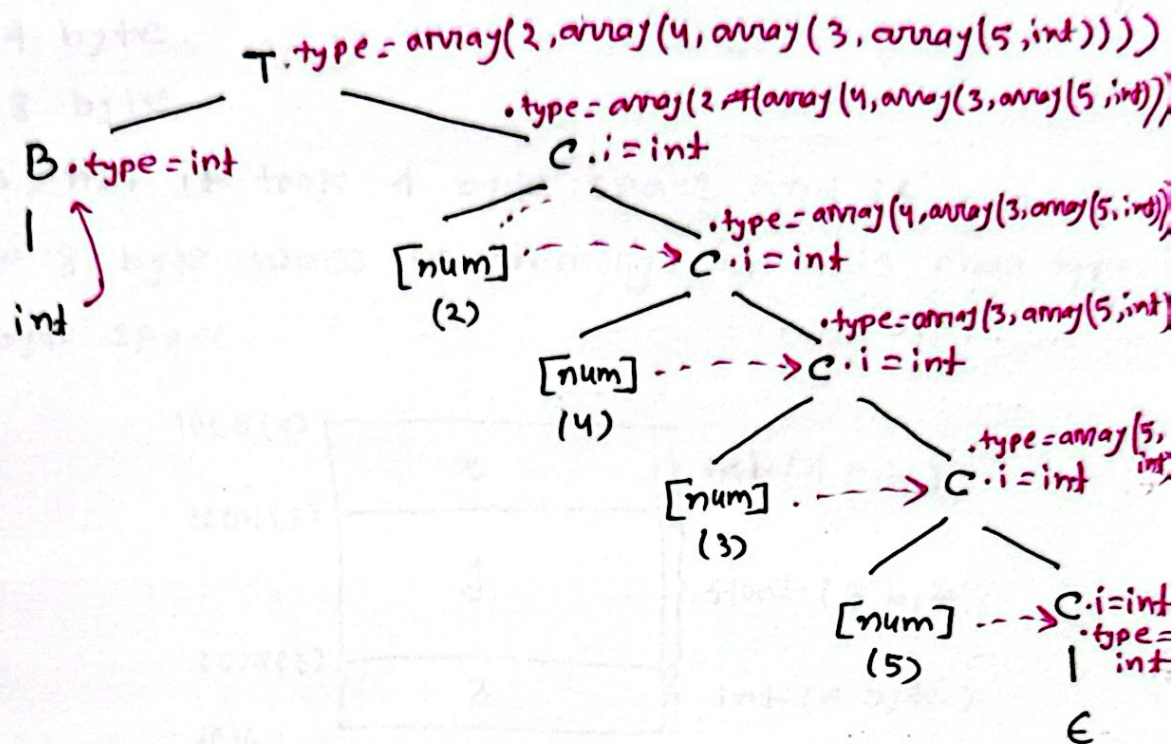
$C \rightarrow \epsilon \{ C.type = C.i \}$

input string:

`int [3] [4]`



Input string: int [2][4][3][5]



* Using global variable

T → B { t = B.type } C { T.type = C.type }

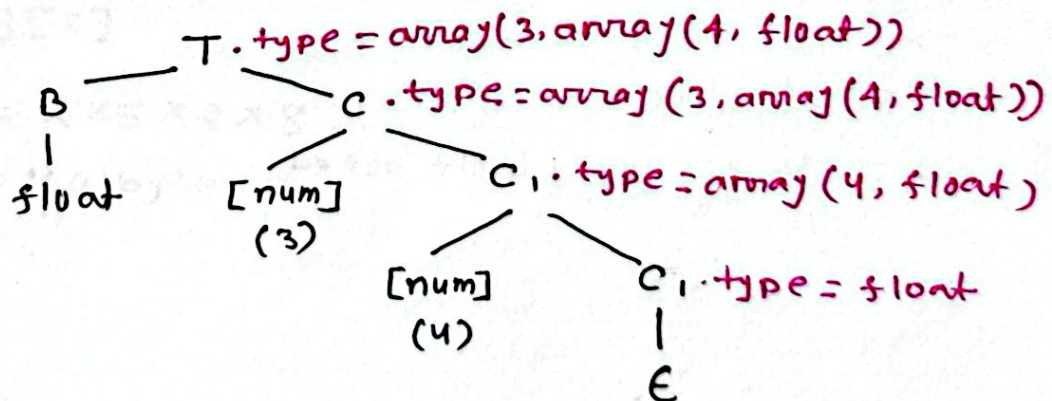
B → int { B.type = int }

B → float { B.type = float }

C → [num] C₁ { C.type = array (num.val, C₁.type) }

C → ε { C.type = t }

Input string: float [3][4]



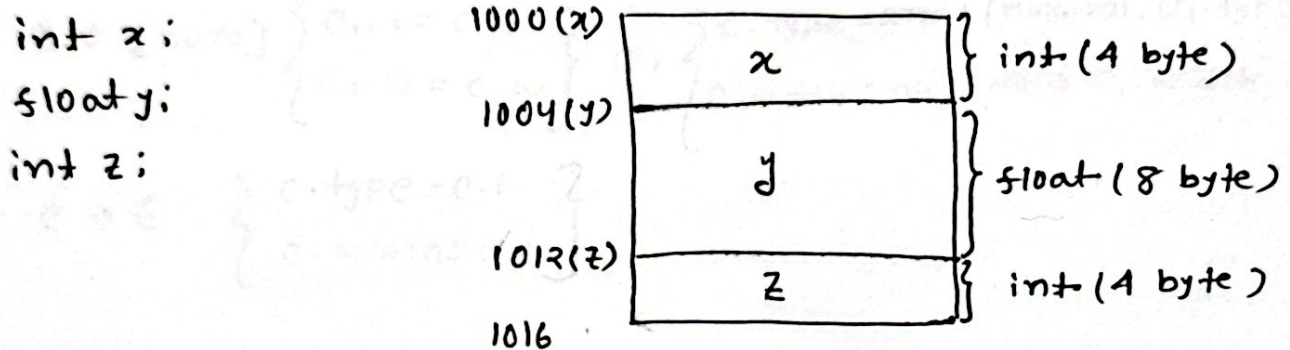
SDT for Width Calculation

Width $\rightarrow$ how much space a particular type taken in memory

int x, $\rightarrow$ 4 byte

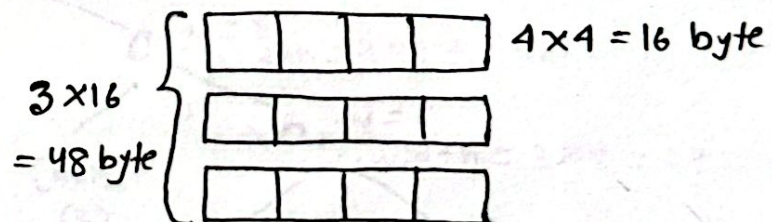
float x, $\rightarrow$ 8 byte

So, if x is int, it took 4 byte space and if x is float, it took 8 byte space in memory. if x is char type, it took 1 byte space.



* int[3][4] $\rightarrow$ 3 array (1 dimensional), each has 4 integer

$\hookrightarrow$ array(3, array(4, int))



* float [2][5][2]

space needed = $2 \times 5 \times 2 \times 8$

= 160 byte

$\hookrightarrow$ for float

*

$$T \rightarrow B \left\{ \begin{array}{l} c.i = B.type \\ c.w = B.width \end{array} \right\} c \left\{ \begin{array}{l} T.type = c.type \\ T.width = c.width \end{array} \right\}$$

$$B \rightarrow int \left\{ B.type = int \right\}$$

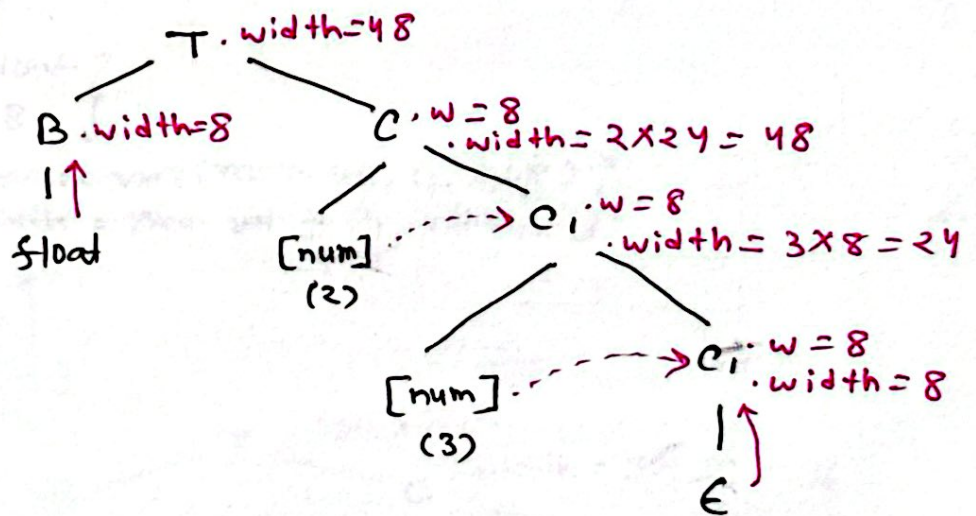
$$B \rightarrow float \left\{ \begin{array}{l} B.type = float \\ B.width = 8 \end{array} \right\}$$

$$c \rightarrow [num] \left\{ \begin{array}{l} c.i = c.i \\ c.w = c.w \end{array} \right\} c_i \left\{ \begin{array}{l} c.type = array(num.val, c_i.type) \\ c.width = num.val * c_i.width \end{array} \right\}$$

$$c \rightarrow \epsilon \left\{ \begin{array}{l} c.type = c.i \\ c.width = c.w \end{array} \right\}$$

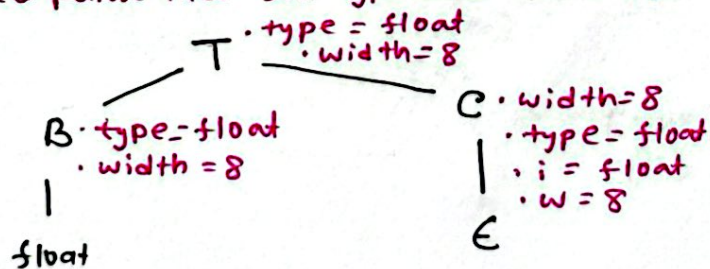
Input string : float [2] [3]

↳ $2 \times 3 \times 8 = 48$ byte



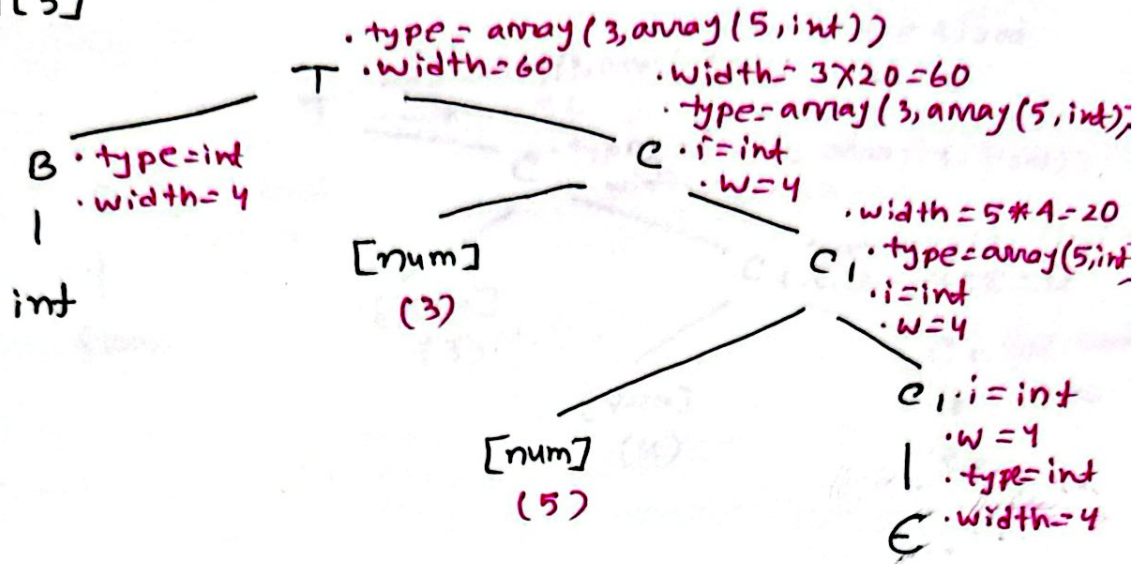
Example 1: Annotated parse tree for Type and Width SDT

Input: float



Example 2

Input: `int [3][5]`



Alternative SDT for width calculation

$T \rightarrow B \left\{ \begin{array}{l} t = B.type \\ w = B.width \end{array} \right\} C \left\{ \begin{array}{l} T.type = C.type \\ T.width = C.width \end{array} \right\}$

$B \rightarrow int \left\{ \begin{array}{l} B.type = int \\ B.width = 4 \end{array} \right\}$

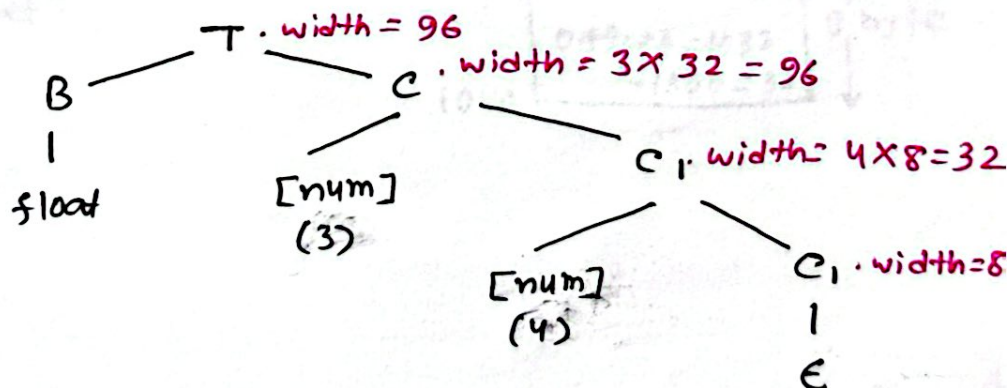
$B \rightarrow float \left\{ \begin{array}{l} B.type = float \\ B.width = 8 \end{array} \right\}$

$C \rightarrow [num] C_1 \left\{ \begin{array}{l} C.type = array(num.val, C_1.type) \\ C.width = num.val * C_1.width \end{array} \right\}$

$C \rightarrow \epsilon \left\{ \begin{array}{l} C.type = t \\ C.width = w \end{array} \right\}$

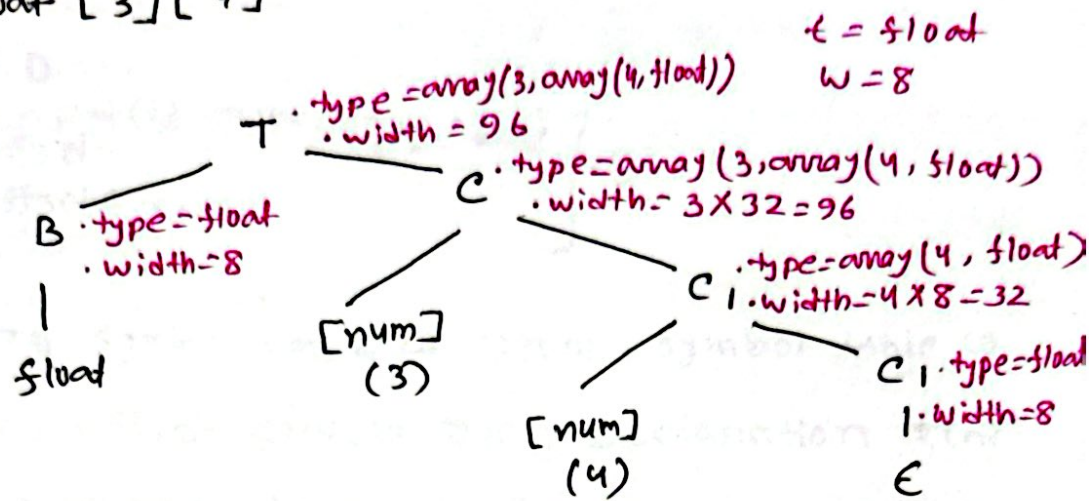
Input: `float [3][4]`

$\hookrightarrow 3 \times 4 \times 8$



Example: Annotated parse tree for alternative width calculation SDT

Input string: float [3][4]

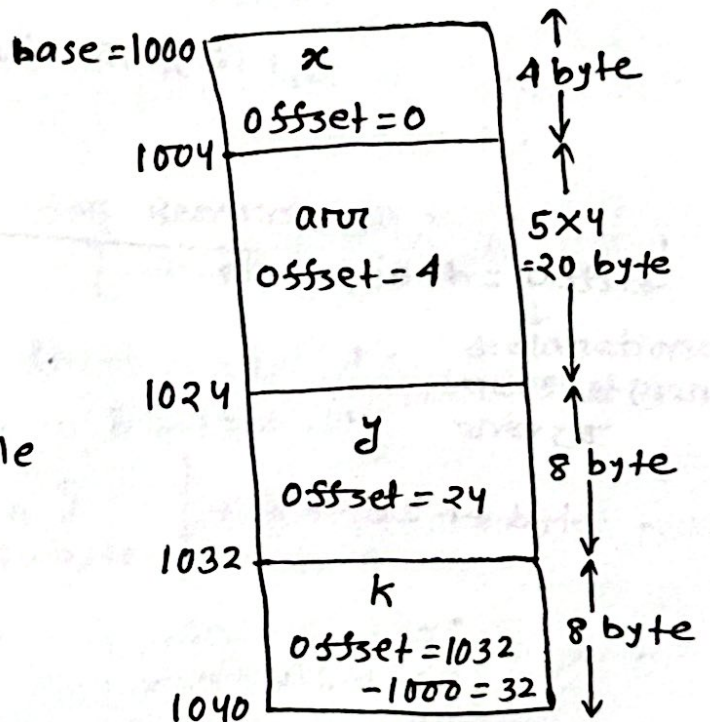


Offset → relative position

```

int x;
int[5] arr;
float y;
float k;
    
```

Memory to code segment (মমানত)
load রাখে, তার- জালক- variable
কমানত আছে, তার relative
position রাখে offset.



SDT of offset calculation for Normal variable and Array variable

$P \rightarrow \{offset = 0\} ID$

$D \rightarrow T \{id; \begin{cases} top.put(id.lexeme, T.type, offset) \\ offset = \\ offset + T.width \end{cases} \} ID$

compilen সবকিছু Symbol table এ রাখা। symbol table এ type, id lexeme, offset রাখতে হয়। Declaration হলে symbol table এ incorporate করতে হয়। So অর্থাৎ symbol table এ store করলে, তারপর offset বাড়ানো। কারণ 'offset = offset + T.width' হবে next declaration এর offset. এইজন্য top.put রাখতে।

Width and offset calculation for Record

```
int a;
record
{

```

int a; $\rightarrow 4 \text{ byte}$

float [5] b; $\rightarrow 5 \times 8 = 40 \text{ byte}$

} P1;

int c;

a, int, 0
P1, record, 4
c, int, 48

a, int, 0
b, float [], 40
c, int, 44
..., ..., 48

VESTAR MR[®]
TRIMETAZIDINE

Nitrin[®] SR
Nitroglycerin USP 2.6 mg & 6.4 mg Tablet

Healthcare

$T \rightarrow \text{recond } \{ \text{Env. Push(top); Stack. Push(offset);}$
 $\text{top} = \text{new Env()}; \text{offset} = 0$
 $D \{ \{ \text{top} = \text{Env. Pop()};$
 $\{ T.\text{width} = \text{offset};$
 $\{ \text{offset} = \text{Stack. Pop()};$
 $\{ T.\text{type} = \text{recond(top)} \}$

$P \rightarrow \{ \text{offset} = 0 \} D$

$D \rightarrow T \text{ id}; \{ \text{top.put(id.lexeme, T.type, offset)} \} D$
 $\{ \text{offset} = \text{offset} + T.\text{width} \}$

$T \rightarrow B \{ \{ c.i = B.\text{type} \} C$
 $\{ c.w = B.\text{width} \} \{ T.\text{type} = c.\text{type}$
 $\{ T.\text{width} = c.\text{width} \}$

$B \rightarrow \text{int} \{ B.\text{type} = \text{int}$
 $\{ B.\text{width} = 4 \}$

$B \rightarrow \text{float} \{ B.\text{type} = \text{float}$
 $\{ B.\text{width} = 8 \}$

$c \rightarrow [num] \{ \{ c.i = c.i$
 $\{ c.w = c.w \} \{ c.\text{type} = \text{array}(num.\text{val}, c.\text{type})$
 $\{ c.\text{width} = num.\text{val} * c.\text{width} \}$

$c \rightarrow \epsilon \{ \{ c.\text{type} = c.i$
 $\{ c.\text{width} = c.w \}$

$T \rightarrow \text{recond } \{ \{ \text{Env. Push(top); Stack. Push(offset);}$
 $\{ \text{top} = \text{new Env()}; \text{offset} = 0 \}$

$\{ \text{top} = \text{Env. Pop()};$
 $\{ T.\text{width} = \text{offset};$
 $\{ \text{offset} = \text{Stack. Pop()};$
 $\{ T.\text{type} = \text{recond(top)} \}$

0.000000
1.000000
8.000000

Annotated parse tree for declaration and Type expression.

int a;

recond }

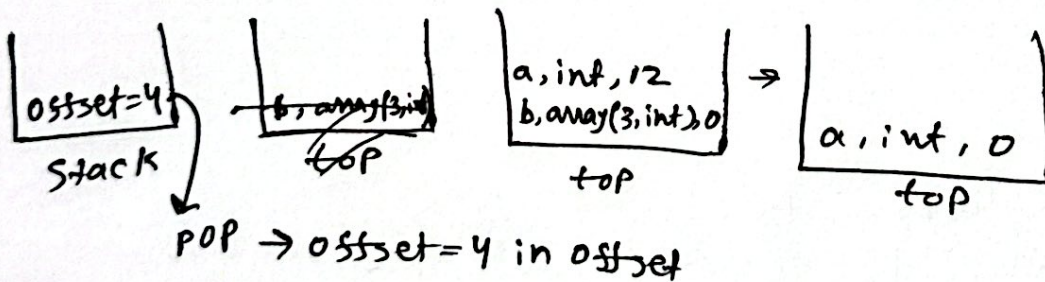
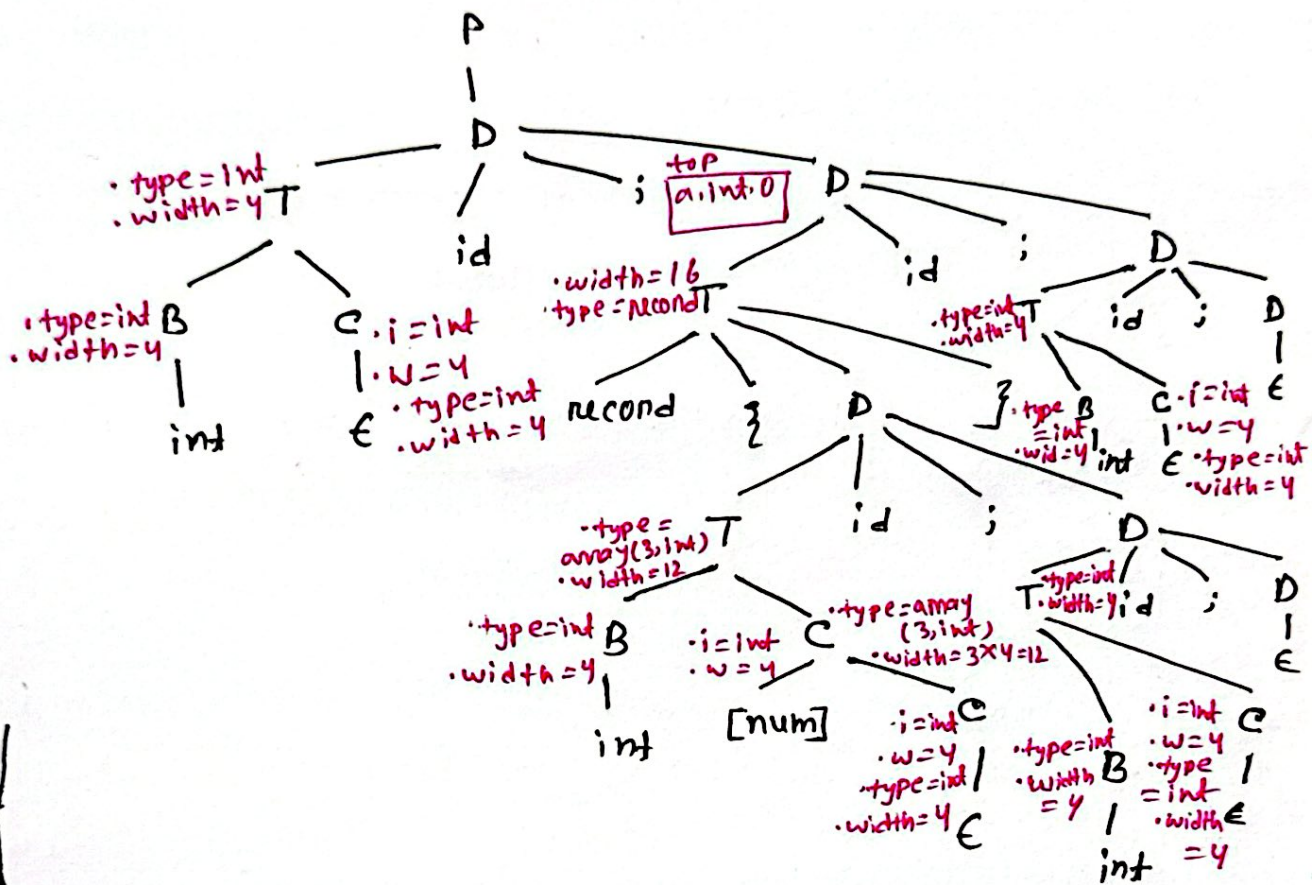
int [3] b;

int a;

} P1;

int c;

offset = 0 4 8 12 16 20 24



Nitrin[®] SR
Nitroglycerin USP 2.6 mg & 6.4 mg Tablet



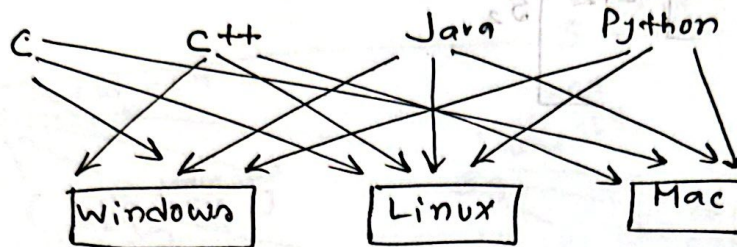
VESTAR MR[®]
TRIMETAZIDINE

TOPIC NAME: Lecture 13

DAY: _____

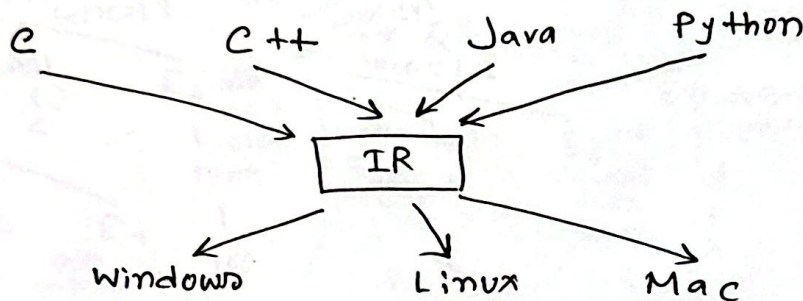
TIME: _____

DATE: / /

Intermediate Code Generation

→ 4x3
= 12 compiler

2 $\begin{bmatrix} 12 \\ 32 \end{bmatrix}$



4 frontend
+ 3 backend

এছাড়া new language introduce করলে অল্প
frontend বানানো লাগতে হবে; new machine introduce
করলে অল্প backend বানানো লাগতে।

IR use না করলে, number of compiler = $m \times n$

IR use করলে, number of frontend = m

+ n backend = n

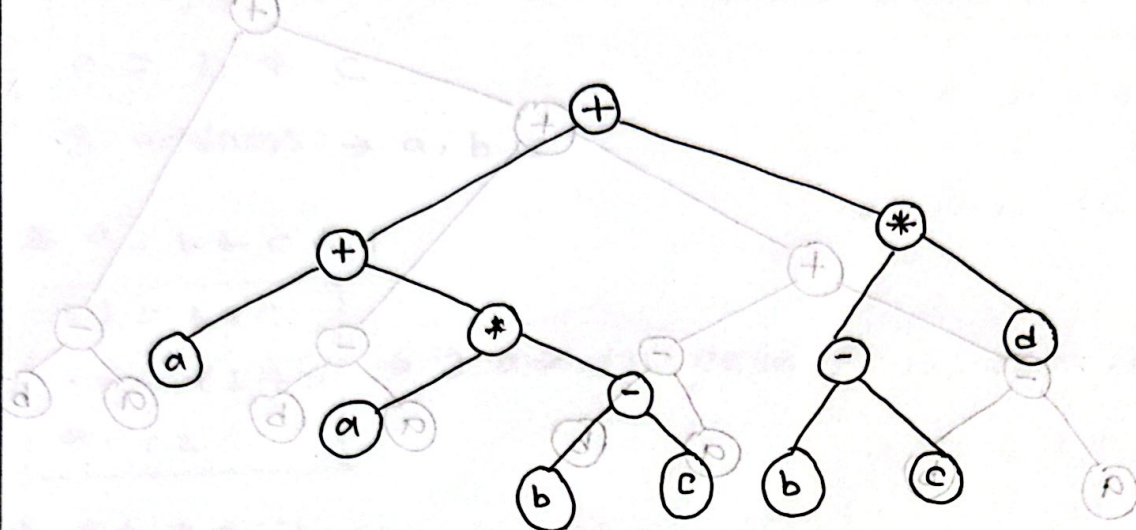
TOPIC NAME : YAD

DAY : _____

TIME : _____ DATE : / /

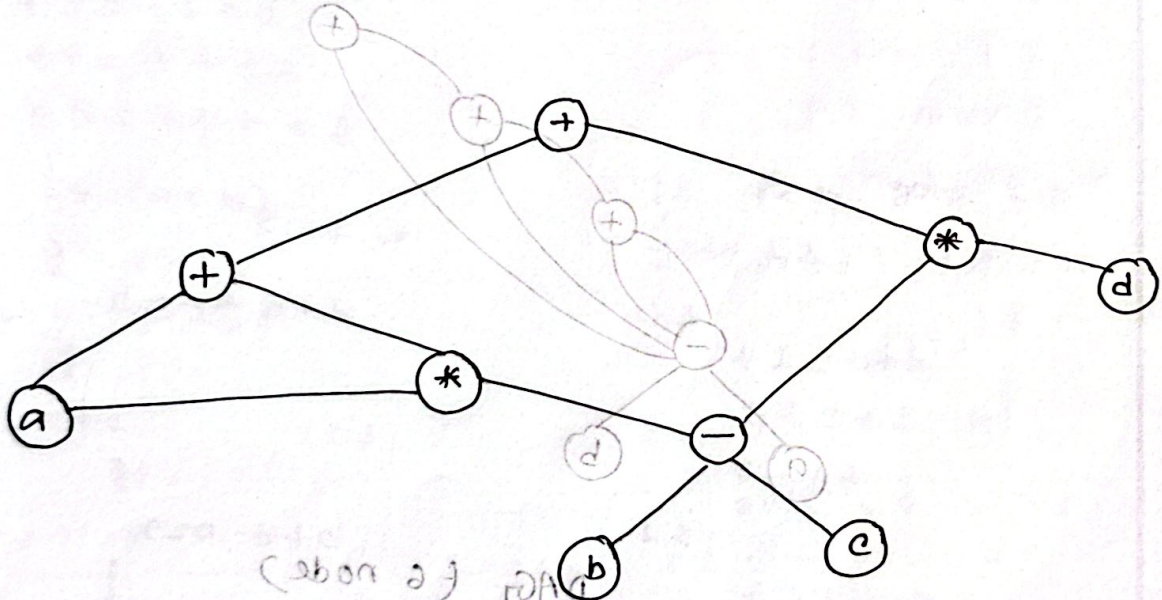
Syntax tree

$$a + a * (b - c) + (b - c) * d$$



syntax tree to redundancy (b-c)

Solution : DAG (Directed Acyclic Graph)



no redundancy, so, memory efficient

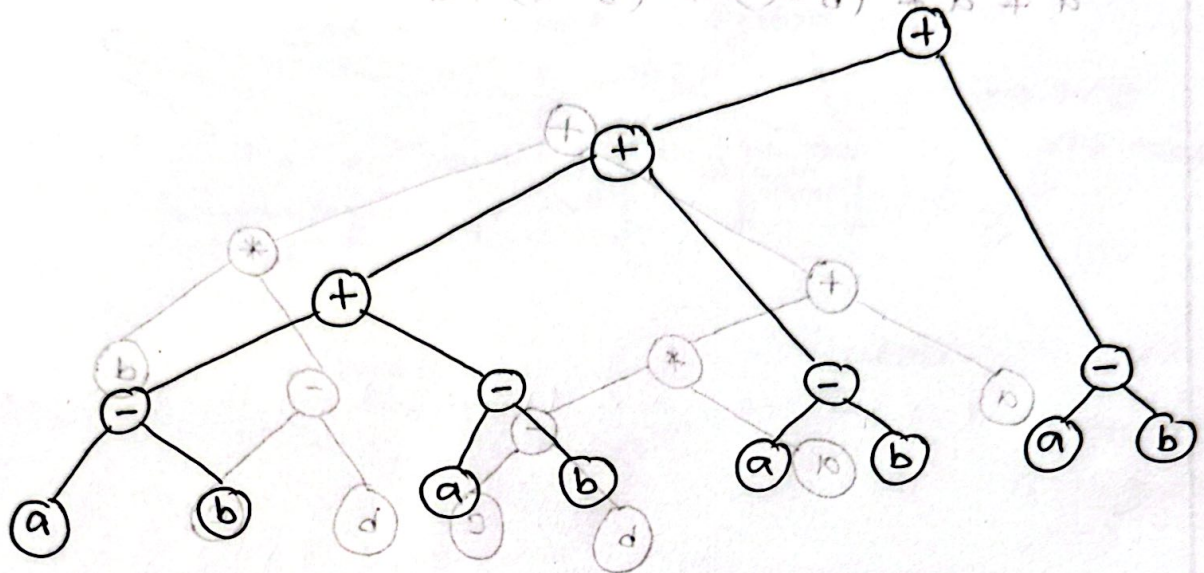
TOPIC NAME : _____

DAY: _____

TIME: _____

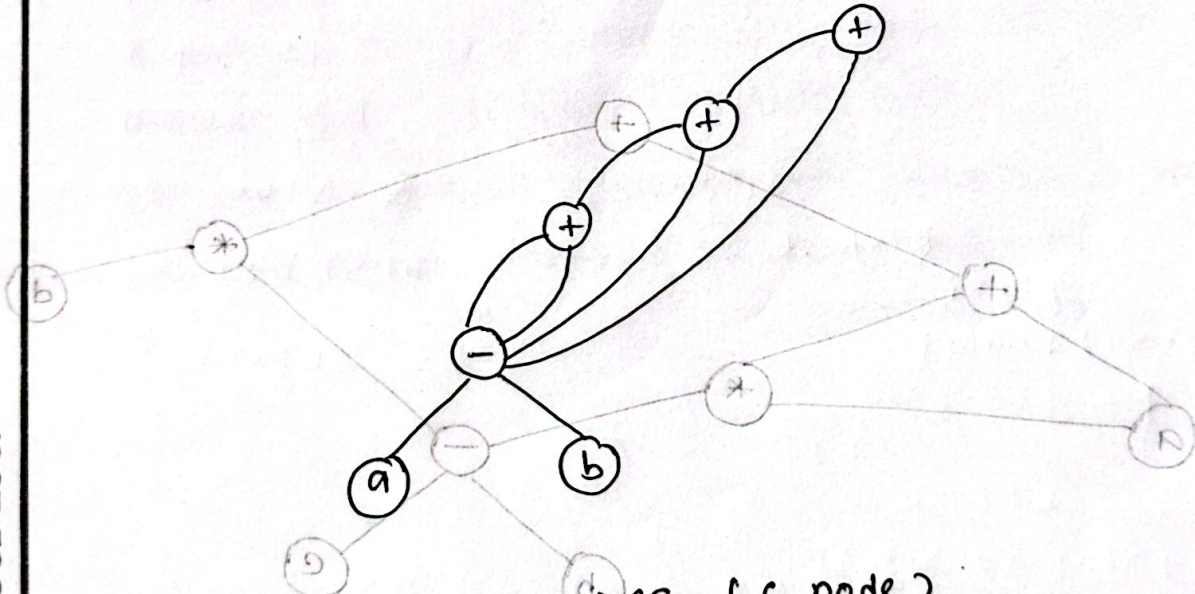
DATE: / /

$$(a-b) + (a-b) + (a-b) + (a-b)$$



syntax tree (15 node)

Directed Acyclic Graph (DAG)



DAG (6 node)

Optimize a DAG use DAG

GOOD LUCK

TOPIC NAME: _____

DAY: _____

TIME: _____ DATE: / /

Three Address Code (TAC)

↳ maximum 3 address variables

$$a = b * c$$

3 address $\rightarrow a, b, c$

$$* a = b * c + d$$

$$t1 = b * c$$

$$t2 = t1 + d$$

$$a = t2$$

$\rightarrow$ 3 address code

$$* a = a * (b - c) + (b - c) * d$$

$$t1 = b - c$$

$$t2 = a * t1$$

$$t3 = t1 * d$$

$$t4 = a + t2$$

$$t5 = t4 + t3$$

$$* \text{ if } (a < b)$$

$$\{ i = i + 1$$

$$c = a * b + k$$

$\}$

else

$$\{ i = i + 1$$

$$c = a - b + c$$

$\}$

$$\text{if } a < b \text{ goto L2}$$

goto L2

L1:

$$t1 = a * b$$

$$t2 = t1 + k$$

$$c = t2$$

goto L3

L2:

$$t3 = a - b + c$$

$$t4 = t3 + c$$

$$c = t4$$

L3:

TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

* func (a, b, c)

param a
param b
param c
call func, 3

* a = func (g * k / 1 + a, e + f)

t1 = g * k
t2 = t1 / 1
t3 = t2 + a
param t3
param c
t4 = e + f
param t4

a = call func, 3

* while (i < 10)

{
i = i + 1;
}

L1:
if i < 10 goto L2
goto L3
L2:
i = i + 1
goto L1
L3:

* do {

i = i + 1;
} while (i < 10);

L1:
i = i + 1
if (i < 10) goto L1

L2:

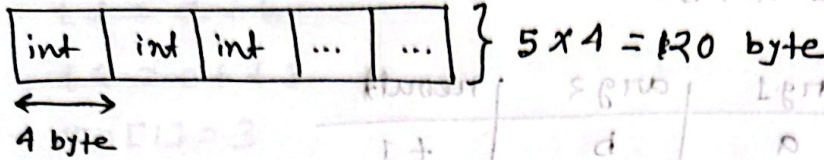
TOPIC NAME : _____

DAY : _____

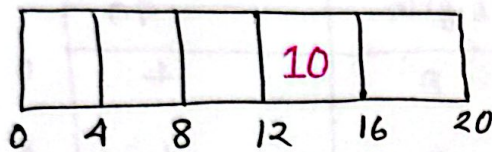
TIME : _____ DATE : / /

int arr[5];

int = 4 byte
float = 8 byte



arr[3] = 10
 ↓
 index



$t1 = 3 * 4$
arr[t1] = 10
3 address code

* a = b * arr[i] + c[j] * 3.5

$t1 = i * 4$
 $t2 = \text{arr}[t1]$
 $t3 = b * t2$
 $t4 = j * 8$
 $t5 = c[t4]$
 $t6 = t5 * 3.5$
 $a = t3 + t6$

$t1 = i * 4$
 $\text{arr}[t1] = 10$

Address	8 bits	16 bits	32 bits
t1	1	1	*
0	1	1	[] =
→ 1000	0	1	= []

Quadruples

$$t_1 = a + b$$

op	arg1	arg2	result
+	a	b	t1

$a = -b$ minus

op	arg1	arg2	result
minus	b	a	

```
if a < b goto L1
goto L2
```

OP	arg1	arg2	result
if <	a	b	21

$$C = \text{array}[i]$$
$$t1 = i * 4$$

$$c = arr[t1]$$

OP	arg 1	arg 2	result
*	i	4	t1
= []	arr	t1	c
[] =	t1	c	arr ←

$$w[1+t] = c$$

TOPIC NAME: _____

DAY: _____

TIME: _____ DATE: / /

Triples → no result column

$$t1 = a + b$$

$$t2 = c + t1$$

$$arr[i] = 3$$

$$x = arr[i]$$

	OP	arg 1	arg 2
0	+	a	b
1	+	c	t1
2	[]	arr	i
3	=		3
4	[]	arr	i
5	=	x	

$$arr[i] = 3$$

$$x = arr[i]$$

$\left\{ \begin{array}{l} \text{if } a < b \text{ goto } L1 \\ \text{goto } L2 \end{array} \right.$
 $\left\{ \begin{array}{l} t1 = a < b \\ \text{if true } t1 \text{ goto } L1 \\ \text{goto } L2 \end{array} \right.$

	OP	arg 1	arg 2
0	<	a	b
1	if true	(0)	L1
2	goto	L2	

TOPIC NAME : YAG

DAY: _____

TIME: _____

DATE: / /

Indirect Triples

ins no

35 (0) $t1 = \text{minus } c$

36 (1) $t2 = b * t1$

37 (2) $t3 = \text{minus } c$

38 (3) $t4 = b * t3$

39 (4) $t5 = t2 + t4$

40 (5) $a = t5$

$\varepsilon = [i] \text{ row}$

	OP	arg 1	arg 2
0	minus	c	
1	*	b	(0)
2	minus	c	
3	*	b	(2)
4	+	(1)	(3)
5	=	a	(4)

$d + b = 1 +$

$1 + 0 = 5 +$

$\varepsilon = [i] \text{ row}$

$[i] \text{ row} = x$

90

+

+

[]

=

[]

=

(2)

(3)

(4)

$d > a = 1 +$

$1 + 0 + 0 + 1 + \text{sum} + 1$

$5 + 0 + 0 + 5$

$\varepsilon \text{ row}$

$\varepsilon \text{ row}$

40

d

a

>

[]

(0)

sum + 1

5

0 + 0 + 5

TOPIC NAME: Quadruple

DAY: _____

TIME: _____

DATE: / /

```

if (a < b)
{
    arr[i] = x * y + z;
}
else
{
    arr[i] = 2;
}
p = arr[i];

```

Quadruple

```

if a < b goto L1
goto L2

```

```

L1:
    t1 = x * y
    t2 = t1 + z
    t3 = i * 4
    arr[t3] = t2
    goto L3

```

```

L2:
    t4 = i * 4
    arr[t4] = 2
L3:
    t5 = i * 4
    p = arr[t5]

```

OP	arg 1	arg 2	result
if <	a	b	L1
goto			L2
*	x	y	t1
+	t1	z	t2
*	i	4	t3
[] =	t2	t3	arr
goto			L3
*	i	4	t4
[] =	t4	2	arr
*	i	4	t5
= []	arr	t5	p

TOPIC NAME: Triple

DAY 8

TIME:

DATE: / /

```

if (a < b)
{
    arr[i] = x * y + z;
}
else
{
    arr[i] = 2;
}
p = arr[i];

```

```

t = a < b ? 1 : 0;
if True (t) goto L1
goto L2
L1:
    t1 = x * y
    t2 = t1 + z
    t3 = i * 4
    arr[t3] = t2
    goto L3
L2:
    t4 = i * 4
    arr[t4] = 2
L3:
    t5 = i * 4
    p = arr[t5]

```

Triple:

	OP	arg 1	arg 2	Label
(0)	<	a	b	
(1)	if True	(0)		L1
(2)	goto	L2		
(3)	*	x	y	
(4)	+	(3)	z	
(5)	*	i	4	
(6)	[]	arr	(5)	
(7)	=	(6)	(4)	
(8)	goto	L3		
(9)	*	i	4	
(10)	[]	arr	(9)	
(11)	=	(10)	(7)	
(12)	*	i	4	
(13)	[]	arr	(12)	
(14)	=	p	(13)	

GOOD LUCK™

TOPIC NAME: _____ DAY: _____
TIME: _____ DATE: / /

Quadruple এর ক্ষেত্রে optimizing compiler instruction সূত্রকে rearrange করতে পারে, there is no side effect.

Quadruple এর extra 'result' column এর জন্য storage বেশি লাগে।

কিন্তু Triple ও এর ক্ষেত্রে storage কম লাগে।

Triple এর ক্ষেত্রে optimizing compiler instruction সূত্রকে rearrange করে না, কারণ complexity arise করে।

Triple এর rearrange করতে না পারার সমস্যাটো সমাধান করে indirect triple. (০) index rearrange করার জন্য instruction এর rearrange হয়।

ins	index
100	(0)
101	(1)
102	(2)
...	...
...	...

rearrange
→

ins	index
100	(0)
101	(1)
102	(2)
...	...
...	...
100	(0)

TOPIC NAME: Lecture 14

DAY:

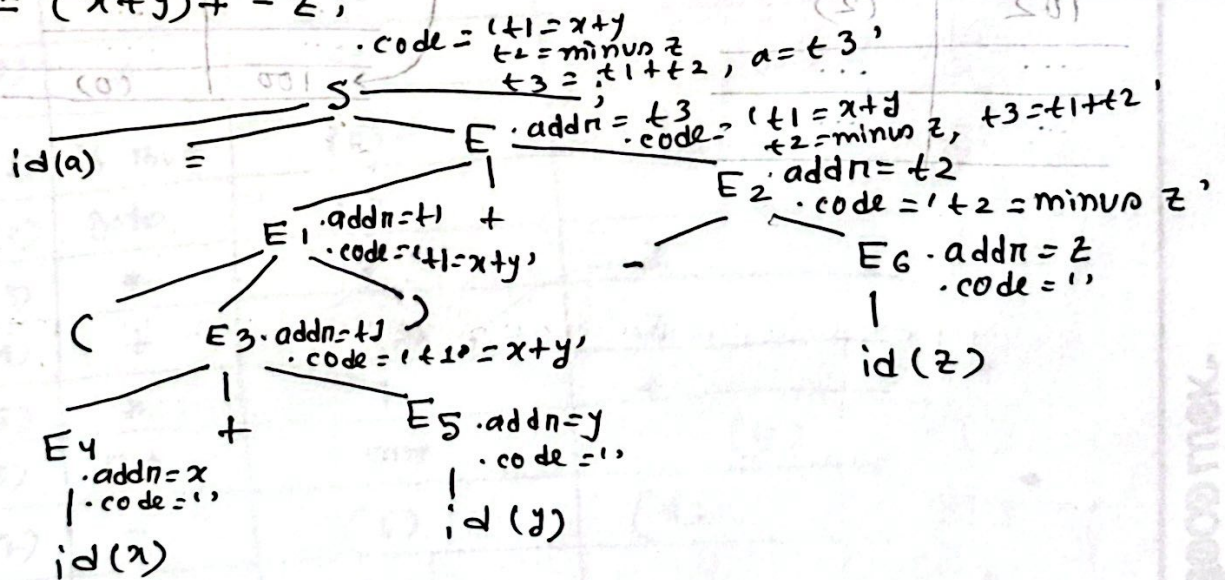
TIME:

DATE: / /

Translation of Normal Expression

Production

Semantic rules

 $S \rightarrow id = E;$
 $S.code = E.code \parallel gen(top.get(id.lexeme)$
 $\quad (=, E.addr))$
 $E \rightarrow E_1 + E_2$
 $E.addr = new Temp()$
 $E.code = E_1.code \parallel E_2.code \parallel gen(E.addr, '=',$
 $\quad E_1.addr, '+', E_2.addr)$
 $E \rightarrow E_1 - E_2$
 $E.addr = new Temp()$
 $E.code = E_1.code \parallel gen(E.addr, '=', 'minus',$
 $\quad E_1.addr)$
 $I(E)$ $E.addr = E_1.addr$ $E.code = E_1.code$ $E.addr = top.get(id.lexeme)$ $E.code = ''$ $a = (x+y) + -z;$ 

On the fly code generation: parse tree structure directly
 three address code to output directly then

TOPIC NAME: _____

DAY: _____

TIME: _____

DATE: / /

$$t_1 = x + y \quad (E_1)$$

$$t_2 = \text{minval } z \quad (E_2)$$

$$t_3 = t_1 + t_2 \quad (E_3)$$

$$a = t_3$$

(5)

→ on the fly

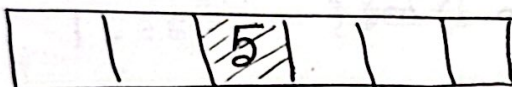
Translation of Array Expression

*

int arr [4];

$$\text{arr}[2] = 5; \rightarrow 1000 + 2 * 4 = 1008$$

base address = 1000



$$1000 \leq 1004 = 1008 \leq 1012 \leq 1016 \leq 1020$$

$$t_1 = 2 * 4$$

$$\text{arr}[t_1] = 5$$

* int arr [3][4];

arr [2][2] = 5;

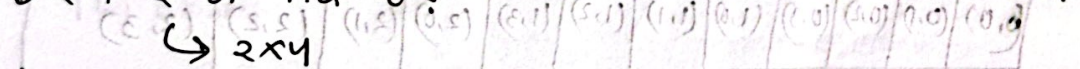
(0,0)	(0,1)	(0,2)	(0,3)
(1,0)	(1,1)	(1,2)	(1,3)
(2,0)	(2,1)	(2,2)	(2,3)

↑
3

1 for 1D array width = $4 \times 4 = 16$ byte

2 for 1D " " = $16 \times 2 = 32$ byte

3 2 + 2 for int 4's



= $32 + 8 = 40$ byte skip

$$(2,2) \text{ address} = 1000 + 40 = 1040$$

↳ base address (given)

GOOD LUCK™

TOPIC NAME: Row major Ordering

DAY: _____

TIME: _____

DATE: / /

3 x 4

array (3, array (4, int))

arr [1][3]

1 for array (4, int) $\rightarrow 1 \times 4 \times 4 = 16$

+ 3 for int $\rightarrow 3 \times 4 = 12$

total = 16 + 12 = 28

* int arr [3][6][4];

(array (3, array (6, array (4, int))))

arr [2][3][1] = 6;

2 for array (6, array (4, int)) $\rightarrow 2 \times 6 \times 4 \times 4 = 192$

+ 3 for array (4, int) $\rightarrow 3 \times 4 \times 4 = 48$

+ 1 for int $\rightarrow 1 \times 4 = 4$

(0,0)	(0,1)	(0,2)	(0,3)
(1,0)	(1,1)	(1,2)	(1,3)
(2,0)	(2,1)	(2,2)	(2,3)

offset = 244

Memory location = base address + 244

1st row				2nd row				3rd row			
(0,0)	(0,1)	(0,2)	(0,3)	(1,0)	(1,1)	(1,2)	(1,3)	(2,0)	(2,1)	(2,2)	(2,3)

1000 1004 1008 1012 1016 1020 1024 1028 1032 1036 1040 1044

row wise memory fillup $\rightarrow$ row major ordering

(row) address

TOPIC NAME: _____

DAY: _____

TIME: _____

DATE: / /

1st column			2nd column			3rd column			4th column		
(0,0)	(1,0)	(2,0)	(0,1)	(1,1)	(2,1)	(0,2)	(1,2)	(2,2)	(0,3)	(1,3)	(2,3)
1000	1004	1008	1012	1016	1020	1024	1028	1032	1036	1040	1044

This is column major ordering.

(1, 3)
↓
column skip
row skip

$$3 \times (3 \times 4) + 1 \times 4 = 36 + 4 = 40 \text{ byte}$$

*

S → id = E;
| L = E;

{ gen(top.get(id.lexeme) '=', E.addr); }

{ gen(L.addr.base '[' L.addr ']' '=', E.addr); }

E → E₁ + E₂

{ E.addr = new Temp();

gen(E.addr '=', E₁.addr '+', E₂.addr); }

| id

{ E.addr = top.get(id.lexeme); }

{ E.addr = new Temp(); }

gen(E.addr '=', L.array.base '[' L.addr '];

{ L.array = top.get(id.lexeme);

L.type = L.array.type.elem;

L.addr = new Temp();

gen(L.addr '=', E.addr '*', L.type.width); }

{ L.array = L₁.array;

L.type = L₁.type.elem;

t = new Temp();

L.addr = new Temp();

gen(t '=', E.addr '*', L.type.width); }

gen(L.addr '=', L₁.addr '+', t); }

TOPIC NAME: _____

DAY: _____

TIME: _____

DATE: / /

$$t_1 = i * 12 \quad (L_1)$$

$\hookrightarrow 3 \times 4$

$$t = j * 4$$

$$t_3 = t_1 + t_2 \quad \left. \vphantom{t_3 = t_1 + t_2} \right] L$$

$$t_4 = a[t_3] \quad (E_2)$$

$$t_5 = c + t_4 \quad (E)$$

$$x = t_5 \quad (S)$$

2x3

$\hookrightarrow \text{array}(2, \text{array}(3, \text{int}))$

$$x = c + a[i][j]$$

$$\begin{array}{c} \downarrow \quad \quad \quad \hookrightarrow j * 4 \\ i * 3 * 4 \end{array}$$

$$\text{So, } i * 3 * 4 + j * 4$$

TOPIC NAME: Lecture 15

DAY: _____

TIME: _____

DATE: / /

Control Flow Statements

if (a > b || a < b && a != b)

! → higher

&&

|| → lowest precedence

a && b || a > b || a < c && a != b

Short circuit code

if (x < 100 || x > 200 && x != y)

x = 0;

Jumping code

if x < 100 goto L₂

if False x > 200 goto L₁

if False x != y goto L₁

L₂: x = 0

L₁:

TOPIC NAME: Lecture 16

DAY: _____

TIME: _____ DATE: / /

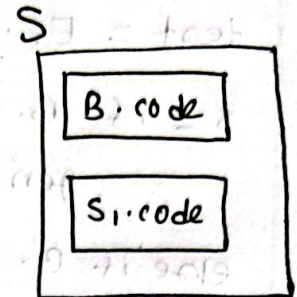
Avoiding Redundant Goto

$S \rightarrow \text{if } (B) S_1$

$B.\text{true} = \text{fall}$

$B.\text{false} = S_1.\text{next} = S.\text{next}$

$S.\text{code} = B.\text{code} \parallel S_1.\text{code}$



L5:

This is a fall through technique.

Because if $B.\text{true}$, we don't need to generate a new level, we simply just fall. So that after $B.\text{code}$, $S_1.\text{code}$ happens.

$B \rightarrow B_1 \parallel B_2$

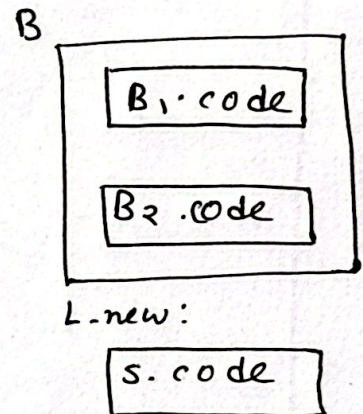
$B_1.\text{false} = \text{fall}$

$B_1.\text{true} = \text{if } B.\text{true} \neq \text{fall}$
then $B.\text{true}$
else $\text{newLabel}()$

$B_2.\text{true} = B.\text{true}$

$B_2.\text{false} = B.\text{false}$

$B.\text{code} = \text{if } B.\text{true} \neq \text{fall}$
then $B_1.\text{code} \parallel B_2.\text{code}$
else $B_1.\text{code} \parallel B_2.\text{code} \parallel \text{label}(B_1.\text{true})$



TOPIC NAME : _____

DAY : _____

TIME : _____

DATE : / /

$B \rightarrow E_1 \text{ rel op } E_2$

test = $E_1 \cdot \text{addr} \text{ rel op } E_2 \cdot \text{addr}$

s = 'if B.true ≠ fail and B.false ≠ fail then

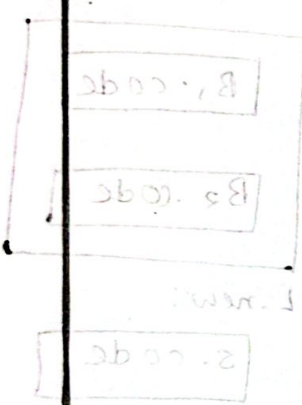
gen('if', test, 'goto', B.true) || gen('goto', B.false)

else if B.true ≠ fail then gen('if', test, 'goto', B.true)

else if B.false ≠ fail then gen('if', test, 'goto', B.false)

else "

B.code = $E_1 \cdot \text{code} || E_2 \cdot \text{code} || s$



$B \rightarrow B_1 || B_2$

$B_1 \cdot \text{false} = \text{fail}$

$B_1 \cdot \text{true} = \text{if } B \cdot \text{true} \neq \text{fail}$

then B.true

else merge(B1)

$B_2 \cdot \text{true} = B \cdot \text{true}$

$B_2 \cdot \text{false} = B \cdot \text{false}$

$B \cdot \text{code} = \text{if } B \cdot \text{true} \neq \text{fail}$

then $B_1 \cdot \text{code} || B_2 \cdot \text{code}$

else $B_1 \cdot \text{code} || B_2 \cdot \text{code} || \text{merge}(B_1 \cdot \text{true})$

GOODLUCK™

TOPIC NAME: Lecture 17 (Handling OOP features)

DAY: _____
TIME: _____ DATE: / /

* `int [5][3] arr;`

$\hookrightarrow$ array (5, array (3, int))

`arr [3][1] = 6;`

$\hookrightarrow$ arr [3x4 + 1x4] = 6

* `int [5][3][5][4] arr;`

`arr [2][1][3][1] = 8;`

`arr [2x3x5x4 + 1x5x4x4 + 3x4x4 + 1x4] = 8`

skip int skip int skip int

[3][5][4] [5][4] [4]

*

`struct student {`

`int studentId;`

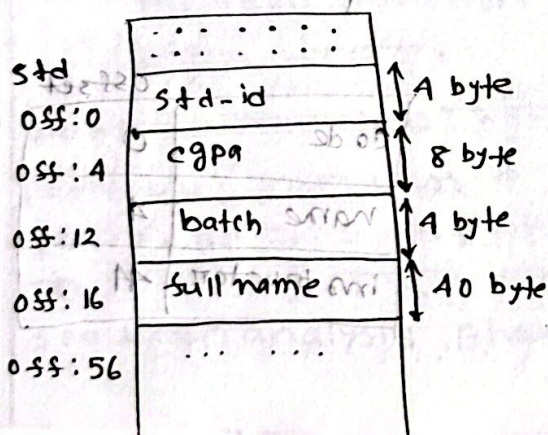
`float cgpa;`

`int batch;`

`char [20] fullName;`

	type	width	offset
std-id	int	4	0
cgpa	float	8	4
batch	int	4	12
full name	array(20, char)	40	16

56



0	std-id
1	cgpa
10	batch
20	fullName
80	

TOPIC NAME: -

YAC

900 (militant) FI 0101501

DAY: -

TIME: -

DATE: / /

student adam;

adam.cgpa = 3.85;

t1 = @adam
t1[4] = 3.85

@adam[4] = 3.85 = [1][1004]

a = [A x 1 + 4 x 2] sum

1000	std-id
	cgpa 3.85

ch = adam.fullname[10] = [1][3][1][5]

↳ index 10

t1 = @adam
t2 = t1[16]
= t2[20]

chan 2 byte
50, 10 x 2 = 20 byte

ch = @adam[36]

struct Teacher {

int employeeId;

char[3] initials;

char[20] name;

char[15] position;

offset

emp-id	0
initials	4
name	10
position	50

--- 80

struct course {

int code;

char[20] name;

Teacher instructor;

code	0
name	4
instructor	11

GOOD LUCK™

TOPIC NAME : _____ DAY: _____

course CS420;

TIME: _____ DATE: / /

initials = cse420.instruction.initials;

t1 = @cs420

t2 = t1[44]

initials = t2[4]

→ initials = @cs420[48]

ch = cs420.instruction.pos[3];
→ 3x2=6

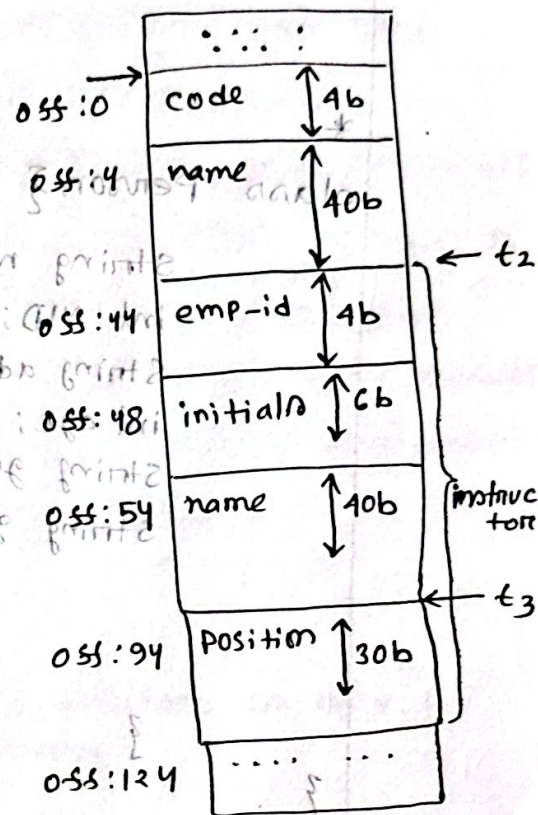
t1 = @cs420

t2 = t1[44]

t3 = t2[50]

ch = t3[6]

→ ch = @cs420[100]



Visibility Control

private: অন্য class থেকে access

করা যায় না, এ class এর

member function শুধু access

করতে পারবে।

public: অন্য class থেকে accessible.

protected: দুই class A & B. B যদি A

র inherit করে, A এর protected variable

র B inherit করতে পারবে।

semantic analysis phase (visibility check করা হয়)

Memory diagram

Student studentA = new Student();
Boolean probation = studentA.isInProbation(threshold);

bool prob = isInProbation(studentA, threshold);

class Person {

String name;

int NID;

String address;

int age;

String getName() {return name;}

String getDescription() {

String desc = name

+ ", " + address;

Return desc;

}

class Student extends Person {

String department;

int studentId;

String getDepartment() {return department;}

String getDescription() {

String desc = name + ", " + studentId + ", " + department;

Return desc;

int getStudentId() {return studentId;}

}

getName	location point
getDesc	

vtable

getStudentId()	person id 3270
getDept()	
getDesc()	
getName()	

vtable

person
id
3270

TOPIC NAME : _____

DAY : _____

TIME : _____ DATE : / /

P. getname() S. getdept()

P. getdesc() S. getdesc()

student এর মধ্যে getDescription না থাকলেও আমরা S. getdesc() call করতে পারবো। কারণ getDescription function টা base class এ exist করে। তখন S. getdesc() call করলে base class এর getDescription call হয়।

student এর মধ্যে getDescription না থাকলে S. getdesc() call করলে student এর নিজস্ব function call হবে। এরকম method টা to overridden method বলা হয়। এখানে parent এর (যে method inherit হয়ে আছে, যেটা আমরা না চেয়ে নিজের implementation চাইছি, অর্থাৎ parent এর function টা আমরা overwrite করছি।

*

P = S;

→ person
~~student~~ এর মধ্যে subclass এর object assign করা হয়েছে।

P. getname()

P. getdesc()

→ student এর getdesc() call হবে, কারণ subclass এর object assign করা হয়েছে।

compiler যেভাবে কাজ করে :

P. getname() call করলে compiler virtual pointer এর through to virtual table এ যাবে। virtual table এ দিয়ে lookup করতে getname কোথায় implemented আছে, then table থেকে (যে location এ চলে যাবে এবং getname() function এর body execute করবে।

GOOD LUCK™

TOPIC NAME :

DAY :

TIME :

DATE :

$P = S$; হলে $P.getdesc()$ call করলে P এর $pointer$ S এর $virtual$ table এর $getdesc$ point করে, so, $student$ এর $getdesc()$ call হবে।

compilation time এ $Object$ এর $actual$ type কি, তা $compiler$ বুঝতে পারে না। $Method$ overwrite হলে তা call হয় runtime এ। একে $dynamic$ dispatch বলা হয়।

← $2 = 9$
অসম্ভব বলা হয়।

() $smarter$. q

() $object$. q

GOODLUCK™

TOPIC NAME: Lecture 19

DAY: _____

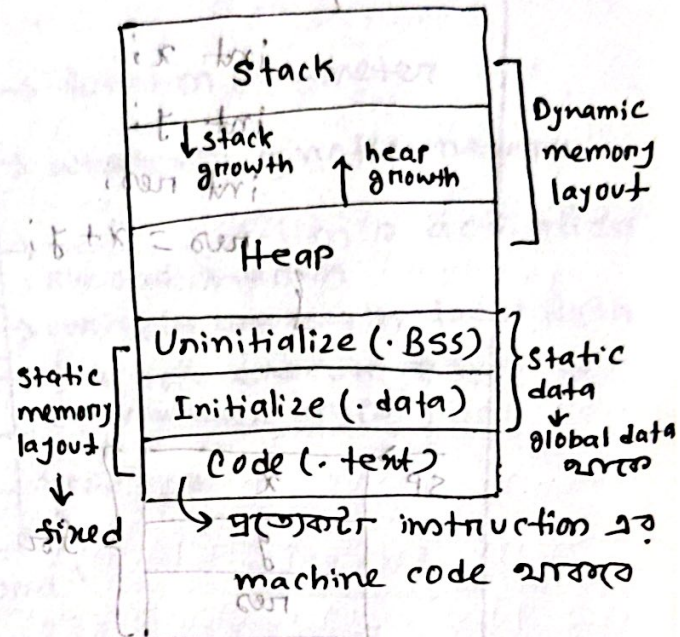
TIME: _____ DATE: / /

Memory Layout of a process

```
int a;
int b = 3;
int main()
{
```

```
    int p = 10;
    int q = 5;
```

```
}
```



যা variable বা node কে dynamically allocate করতে, সব Heap এর মাধ্যমে থাকবে।

function chain আবার call করলে তখন memory

বাড়ছে, যখন function execution শেষ হবে, তখন memory আবার অবস্থান চলে আসবে। তাই stack and heap dynamic memory.

code and data → static

variable dynamically allocate না করলে overwrite হতে পারে, যখন recursion use করতে, তখন এর ফলে আসবে। so same function যার যার call হলেও আমরা memory থেকে ভাগ্য নিজে, একই stack memory গঠন করে।

TOPIC NAME: _____

DAY: _____

TIME: _____

DATE: / /

*

int foo()

{

int x;

int y;

int res;

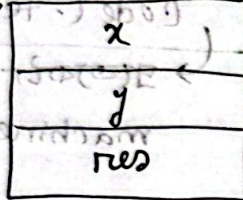
res = x + y;

}

Uninitialized (BSS)

Initialized (data)

SP



offset

x		0
y		4
res		8

int x = 0;

int y = 4;

આખરે y નું address જાણતો રહ્યો ત્યારબાદ, symbol table માં y નું offset શોધી, then sp નું address + offset કરવાને y નું address મળે છે.

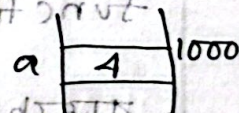
*

int a = 4;

printf("Value of a is: ");

printf("%d", &a);

for address (1000)



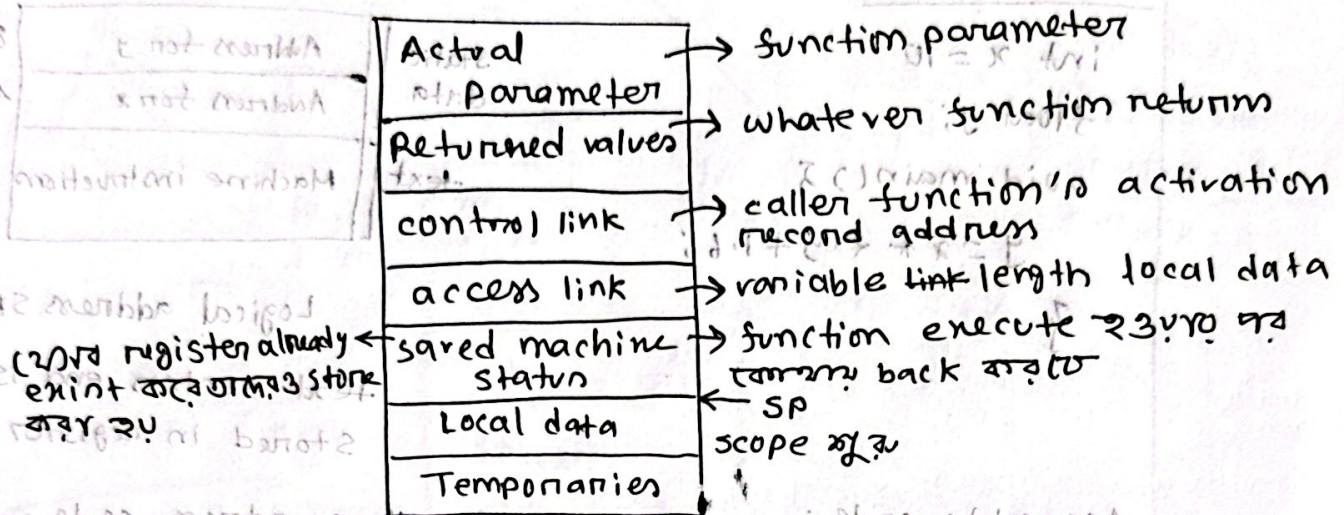
virtual address space માં

1000 location

Logical address

TOPIC NAME : _____ DAY : _____
TIME : _____ DATE : / /

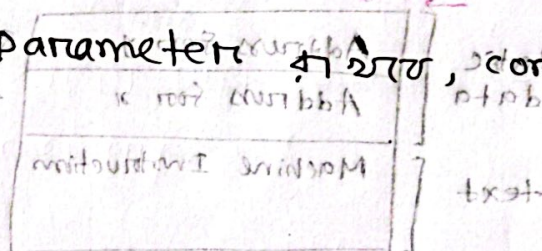
Activation record : function related record store



Activation record

SP ta local data 23y0 ya point 23y0 ya
→ directly symbol table 23y0 ya offset 23y0 ya calculate 23y0 ya

caller responsibility : parameter 23y0 ya, control link



TOPIC NAME: _____

DAY: _____

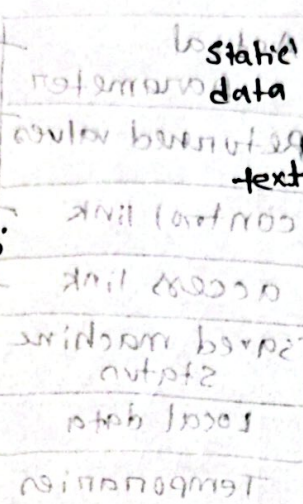
TIME: _____

DATE: / /

Example of Global variable address assignment

Source code:

```
int x = 10;
float y;
void main() {
    y = x * 2.5 + 1.6;
}
```



Address for y	8 byte
Address for x	4 byte
Machine instructions	

Logical address space
text section end is
stored in register \$50

Assembly code:

```
lw $s1, $50[0]
```

```
mult $t0, $s1, 2.5
```

```
addi $t1, $t0, 1.6
```

```
sdw $t1, $50[4]
```

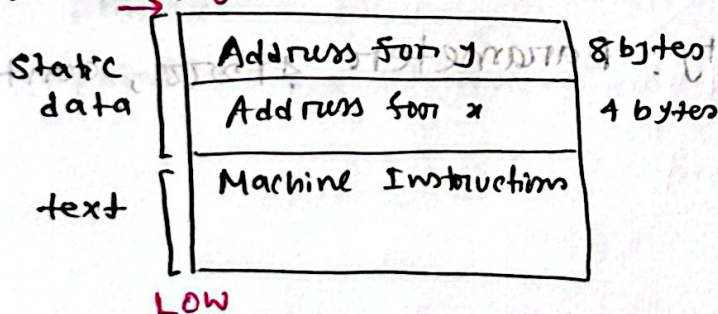
Three address code:

$t0 = x * 2.5$

$t1 = t0 + 1.6$

$y = t1$

* $\$50$ High



Assembly code:

```
lw $s1, $50[-12]
```

```
mult $t0, $s1, 2.5
```

```
addi $t1, $t0, 1.6
```

```
sdw $t1, $50[-8]
```

TOPIC NAME Activation tree

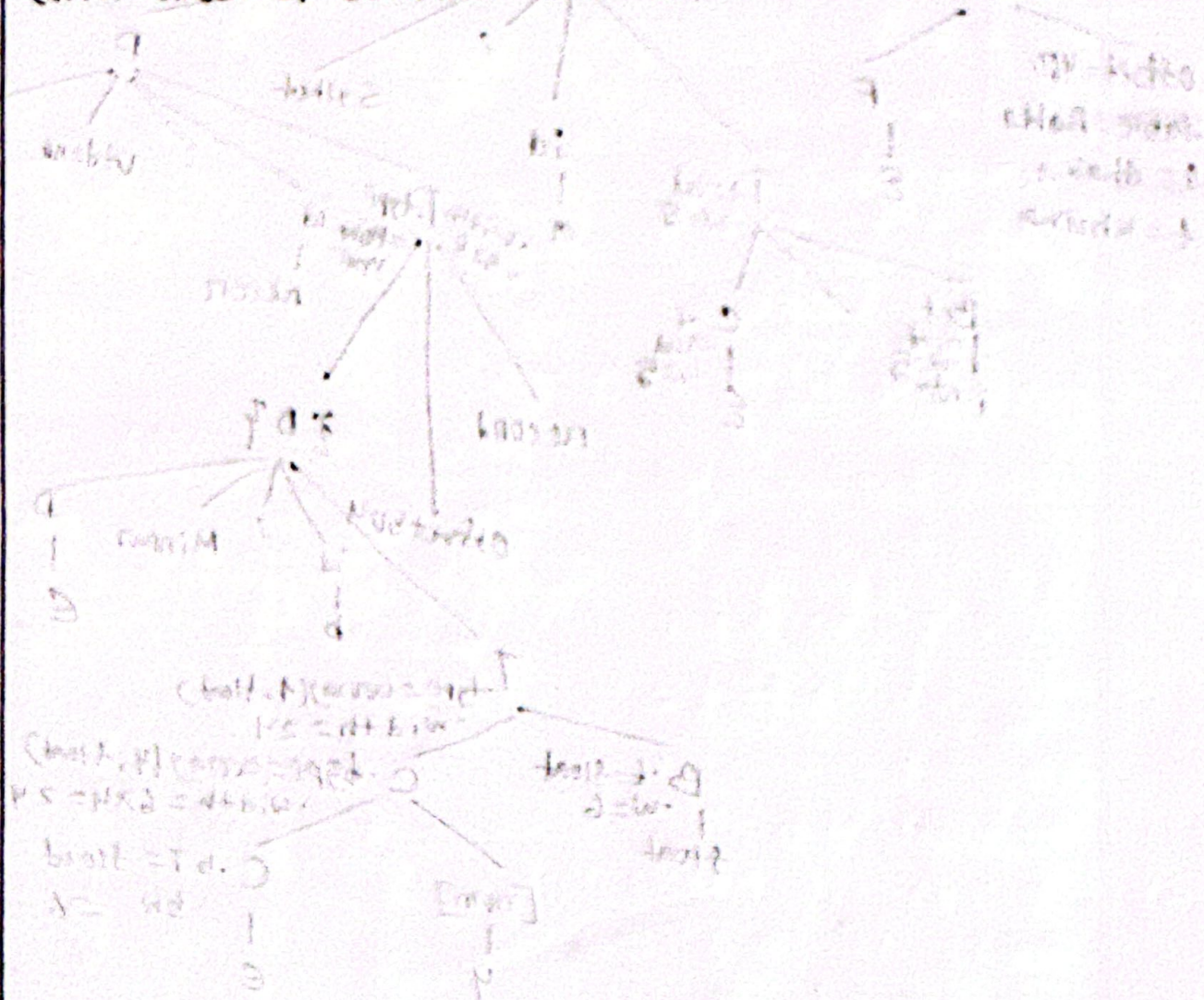
DAY

TIME

DATE _____

तब तो order 1 function exist करता $\rightarrow$ post order traversal

turn order 4 function call $\Rightarrow$ pre order traversal



Only GOOD LUCK™